

Anomaly detection e localizzazione in video termici ferroviari

Relazione tecnica completa del lavoro svolto, degli esperimenti, dei risultati provvisori e del codice sviluppato

Studente	Michele
Documento	Relazione di avanzamento tecnico
Versione	1.0
Data	3 luglio 2026
Progetto	tesi_anomaly_detection
Stato	E01-E03 eseguiti; E04 preparato e non ancora eseguito

Nota di lettura

Obiettivo scientifico vincolante. Il lavoro riguarda l'*anomaly detection/localization*, non la sola object detection. Grounding DINO può proporre regioni, mentre SAM2 può segmentarle e seguirle; nessuno dei due stabilisce automaticamente se un evento sia anomalo. Il primo esperimento che apprende esplicitamente la normalità è E04, già preparato ma non ancora eseguito.

Questo documento descrive soltanto attività, dati e risultati realmente presenti nel progetto alla data indicata. Le metriche E01 ed E02 sono dichiarate provvisorie perché alcune track CVAT devono essere corrette. Non sono stati inventati risultati per E04.

Indice

1. Contesto, obiettivo e definizioni

- 1.1 Anomaly detection vs object detection
- 1.2 Ruolo di CVAT, Grounding DINO e SAM2

2. Organizzazione del progetto

- 2.1 Struttura della repository
- 2.2 Riproducibilità e protezione dei dati

3. Dati video e inventario

4. Annotazione CVAT e ground truth

- 4.1 Protocollo
- 4.2 Export e conversione
- 4.3 Problemi di qualità rilevati

5. Esperimento E01 — SAM2 con box manuale

6. Esperimento E02 — Grounding DINO → SAM2

7. Esperimento E03 — sensibilità al prompt

8. Passaggio alla vera anomaly detection

9. Preparazione E04 one-class

- 9.1 Costruzione dello split
- 9.2 Metodo previsto
- 9.3 Metriche previste

10. Spiegazione precisa del codice Python

- 10.1 Script per video e clip
- 10.2 Script CVAT e visualizzazione
- 10.3 Metriche e valutazione
- 10.4 Configurazioni e stub SAM2
- 10.5 build_anomaly_dataset.py

11. Spiegazione dei notebook Colab

12. Test, controlli e artefatti salvati

13. Limiti attuali e rischi metodologici

14. Prossimi passi ordinati

15. Materiale utilizzabile nella tesi e nelle slide

16. Riferimenti essenziali

1. Contesto, obiettivo e definizioni

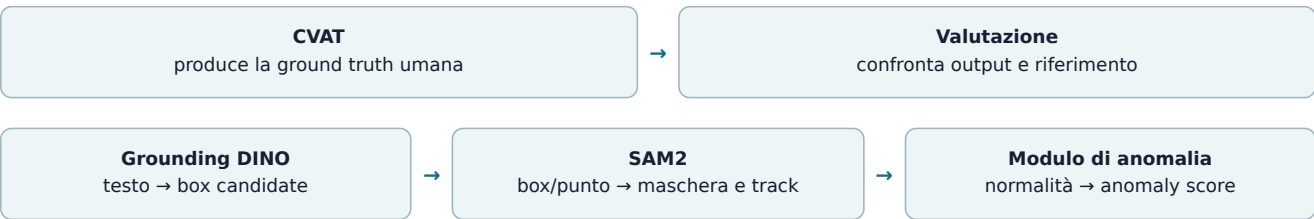
Il dominio applicativo è la sorveglianza di aree ferroviarie tramite telecamere termiche. Nei video possono comparire persone, pallet, scatole, assi o altri elementi che non dovrebbero trovarsi sui binari o nelle aree di sicurezza. La label principale fissata per la ground truth è `anomaly`. Il tipo dell'elemento può essere registrato come attributo descrittivo, ma non deve trasformare il problema in una classificazione supervisionata di categorie note.

1.1 Anomaly detection vs object detection

Compito	Domanda	Output	Uso nel progetto
Object detection	“Che oggetto è e dove si trova?”	Classe, score e bounding box	Solo modulo ausiliario o baseline
Anomaly detection	“Quanto questo frame o questa regione si discosta dalla normalità?”	Anomaly score, decisione normale/anomalo, eventuale heatmap	Obiettivo scientifico finale
Anomaly localization	“In quale zona del frame si trova lo scostamento?”	Heatmap, punto, box o maschera	Obiettivo associato alla detection

Una persona riconosciuta non è automaticamente un'anomalia: dipende dalla posizione e dal contesto. Simmetricamente, un'anomalia sconosciuta potrebbe non appartenere a nessuna classe prevista da un detector. Per questo E01, E02 ed E03 sono esperimenti utili di segmentazione, tracking e proposta di regioni, ma non costituiscono da soli una soluzione completa di anomaly detection.

1.2 Ruolo di CVAT, Grounding DINO e SAM2



- **CVAT** non è un modello: è lo strumento con cui si annota la risposta corretta.
- **Grounding DINO** è stato usato come detector open-vocabulary per ottenere box da prompt testuali.
- **SAM2** è stato usato come segmentatore e tracker video promptable.
- **E04** usa soltanto frame normali per costruire una memoria della normalità e produrre anomaly score.

2. Organizzazione del progetto

È stata creata la cartella di lavoro `tesi_anomaly_detection`. La parola “repository” indica principalmente una struttura ordinata e riproducibile: non significa che i video siano stati pubblicati online. Git è inizializzato localmente, ma i dati sensibili e i risultati pesanti sono esclusi dal versionamento.

2.1 Struttura della repository

```
tesi_anomaly_detection/
├── README.md           guida operativa principale
├── requirements.txt     dipendenze leggere
├── configs/            parametri, prompt e protocollo E04
├── data/
│   ├── raw/           copie di lavoro dei video originali
│   ├── clips/         clip brevi
│   ├── exports_cvat/  export CVAT
│   └── processed/     dataset derivati riproducibili
├── docs/              protocollo, diario, esperimenti e relazione
├── notebooks/         notebook Colab E01, E02 ed E04
├── src/               script Python riutilizzabili
├── tests/             test delle funzioni principali
├── results/
│   ├── inventory/     inventario dei video
│   ├── annotations/   box CVAT convertite
│   ├── metrics/       tabelle quantitative
│   └── qualitative/   immagini, video, maschere e metadati
```

2.2 Riproducibilità e protezione dei dati

- I video originali non vengono modificati dagli script.
- `data/raw/`, `export`, `clip`, `frame`, `risultati`, `checkpoint` e `ZIP` sono ignorati da Git.
- Le configurazioni leggere rimangono in YAML e i risultati in CSV/JSON.
- I modelli pesanti non sono inseriti nei requisiti principali e vengono usati in Colab in un ambiente separato.
- È stato creato il backup locale `tesi_backup_2026-07-03_17-07.tar.gz`.

I dati del relatore sono stati caricati su servizi cloud durante i pilot operativi. In una versione finale della tesi va riportata e rispettata l'autorizzazione ricevuta, evitando qualunque pubblicazione dei video.

3. Dati video e inventario

I tre video sono stati copiati in `data/raw/` e controllati con OpenCV. Tutti risultano leggibili, con codec H.264 e risoluzione 640×512 pixel.

Video	Durata	FPS	Frame	Risoluzione	Codec
ir_2025-01-31_04-15-15.mp4	299,000 s	25,000	7.475	640×512	h264
ir_2025-01-31_05-09-12.mp4	298,000 s	25,000	7.450	640×512	h264
test_video_san_donato.mp4	263,011 s	24,99899	6.575	640×512	h264

Il file prodotto è `results/inventory/inventario_video.csv`. L'inventario permette di controllare durata, frequenza, risoluzione e leggibilità prima di estrarre frame o impostare annotazioni. Il primo video non è ancora stato annotato e rimane disponibile come terzo task/possibile test aggiuntivo.

4. Annotazione CVAT e ground truth

4.1 Protocollo

In CVAT Online è stato creato un progetto con tre task, uno per video. La label principale è `anomalia`. Gli attributi proposti sono:

- `tipo`: persona, pallet, scatola, legno, oggetto generico, altro, incerto;
- `visibilita`: chiaro, parziale, difficile;
- `posizione`: sui binari, vicino ai binari, fuori ROI.

Per una track video si disegna una box sul soggetto, si aggiungono keyframe quando posizione o forma cambiano e si usa `outside` quando l'oggetto scompare. `occluded` segnala invece una visibilità parziale. CVAT interpola automaticamente le box

tra due keyframe: questa funzione riduce il lavoro, ma richiede un controllo visivo perché un'interpolazione può non seguire correttamente il soggetto.

4.2 Export e conversione

Sono stati completati ed esportati in formato CVAT for video 1.1 due task:

Video annotato	Track	Righe box interpolate	Frame con annotazioni	Intervallo
ir_2025-01-31_05-09-12.mp4	6	18.830	6.450	1.000-7.449
test_video_san_donato.mp4	14	24.737	4.847	1.728-6.574

Gli XML sono stati ispezionati, quindi convertiti nei CSV: `ir_2025-01-31_05-09-12_boxes.csv` e `test_video_san_donato_boxes.csv`. Ogni riga contiene video, frame, ID della track, label, coordinate `xtl`, `ytl`, `xbr`, `ybr`, `outside` e `occluded`.

4.3 Problemi di qualità rilevati

Il controllo ha evidenziato che alcune track sono state create con pochi keyframe, prima che fosse chiaro che le annotazioni sarebbero servite come ground truth quantitativa. In particolare, nella clip San Donato tra circa 98 e 101 secondi la box interpolata della persona è spostata rispetto al soggetto reale. Di conseguenza le metriche E01 ed E02 penalizzano anche predizioni visivamente corrette.



Figura 1 — Frame di confronto attorno alla zona critica: la ground truth CVAT richiede revisione. Questo esempio mostra perché la qualità delle annotazioni è parte integrante della valutazione.

Prima di presentare metriche definitive bisogna correggere le track CVAT, soprattutto nell'intervallo 98-101 secondi, e annotare il terzo video.

5. Esperimento E01 — SAM2 con box manuale

Pilot completato — metriche provvisorie

Video	test_video_san_donato.mp4
Clip	85-105 secondi; frame sorgente 2.125-2.624
Dimensione	500 frame, circa 25 fps, 640×512
Modello	SAM 2.1 Small
Checkpoint	sam2.1_hiera_small.pt
Hardware	Google Colab, NVIDIA Tesla T4
Fine-tuning	No
Prompt	Box manuale al frame 0: [392, 195, 438, 270]

È stata preparata localmente la clip `san_donato_pilot_85_105.mp4`. In Colab è stato installato SAM2 dal repository ufficiale ed è stato scaricato il checkpoint Small. La prima box era troppo ampia e produceva una maschera che includeva regioni estranee; è stata quindi ristretta alla sola persona centrale. SAM2 ha propagato la maschera per tutti i 500 frame.



Figura 2 — Campioni ai frame 0, 100, 200, 300, 400 e 499. La maschera segue la stessa persona durante i cambiamenti di postura.

5.1 Risultato qualitativo

- L'identità della persona centrale viene mantenuta per tutta la clip.
- La maschera si adatta ai cambiamenti di postura.
- Il tracking non passa alle altre persone visibili.
- Nel primo frame rimane un piccolo componente spurio disconnesso sopra il soggetto.

5.2 Valutazione provvisoria rispetto a CVAT

La maschera SAM2 è stata trasformata nella box minima che la contiene e confrontata con la track CVAT 1 sui 490 frame sorgente comuni (2.135-2.624).

Soglia IoU	TP	FP	FN	Precision	Recall	F1
0,30	408	82	82	0,833	0,833	0,833
0,50	350	140	140	0,714	0,714	0,714

IoU media: **0,587**; IoU mediana: **0,662**; minimo 0,040; massimo 0,965. Questi valori non sono definitivi a causa dello spostamento della ground truth.

E01 dimostra la capacità di segmentazione e tracking quando è disponibile un prompt geometrico corretto. Non dimostra il rilevamento autonomo di anomalie, perché la box iniziale è manuale.

6. Esperimento E02 — Grounding DINO → box → SAM2

Pilot completato — metriche provvisorie

E02 automatizza la box iniziale tramite il prompt testuale `person`. Grounding DINO Tiny viene eseguito sul primo frame; le box con score almeno 0,30 vengono passate a SAM2.

Detector	IDEA-Research/grounding-dino-tiny
Prompt	person
Soglie	detection 0,20; text 0,15; selezione 0,30
Tracker	SAM 2.1 Small
Fine-tuning	No
Hardware	Google Colab, NVIDIA Tesla T4

6.1 Detection iniziale

- persona al bordo sinistro: score 0,771;
- persona centrale: score 0,631;
- regione debole sullo sfondo: score 0,224, eliminata dalla soglia 0,30.

Box Grounding DINO passate a SAM2



Figura 3 — Le due box con score $\geq 0,30$ usate come prompt geometrici per SAM2.

SAM2 ha propagato entrambe le identità per 500 frame senza scambiarle. Tuttavia una terza persona entra successivamente e non viene scoperta, perché il detector testuale è eseguito soltanto al frame iniziale.



Figura 4 — Tracking delle due persone inizializzate al frame 0. La persona che entra dopo non riceve una nuova identità.

6.2 Matching provvisorio con le track CVAT

Oggetto predetto	Track CVAT	Frame comuni	IoU media	IoU mediana	Frame IoU≥0,3	Frame IoU≥0,5
1	3	227	0,542	0,550	227	133
2	1	490	0,579	0,650	402	349

L’oggetto 1 è visibile per 273 frame prima dell’inizio della track CVAT 3: ciò mostra una lacuna nella ground truth. Lo score di Grounding DINO appartiene esclusivamente al frame iniziale e non deve essere interpretato come confidenza per tutti i frame propagati.

E02 dimostra che un detector testuale può fornire prompt a SAM2. Rimane una pipeline di object proposal e tracking: non assegna ancora un anomaly score rispetto alla normalità della scena.

7. Esperimento E03 — sensibilità al prompt

Pilot del modulo ausiliario completato

Sul frame sorgente 2.125 di San Donato sono stati confrontati sei prompt mantenendo identici modello e soglie. Lo scopo è misurare la sensibilità semantica di Grounding DINO, non valutare una soluzione finale di anomaly detection.

Prompt	Rilevamenti $\geq 0,20$	Selezionati $\geq 0,30$	Score massimo	Tempo
person	3	2	0,771	3,09 s*
box	3	3	0,478	0,38 s
pallet	6	4	0,487	0,38 s
obstacle	7	5	0,508	0,38 s
foreign object	8	6	0,590	0,38 s
anomaly	6	2	0,558	0,38 s

* Il primo tempo include warm-up e inizializzazione; non prova che `person` sia intrinsecamente più lento.

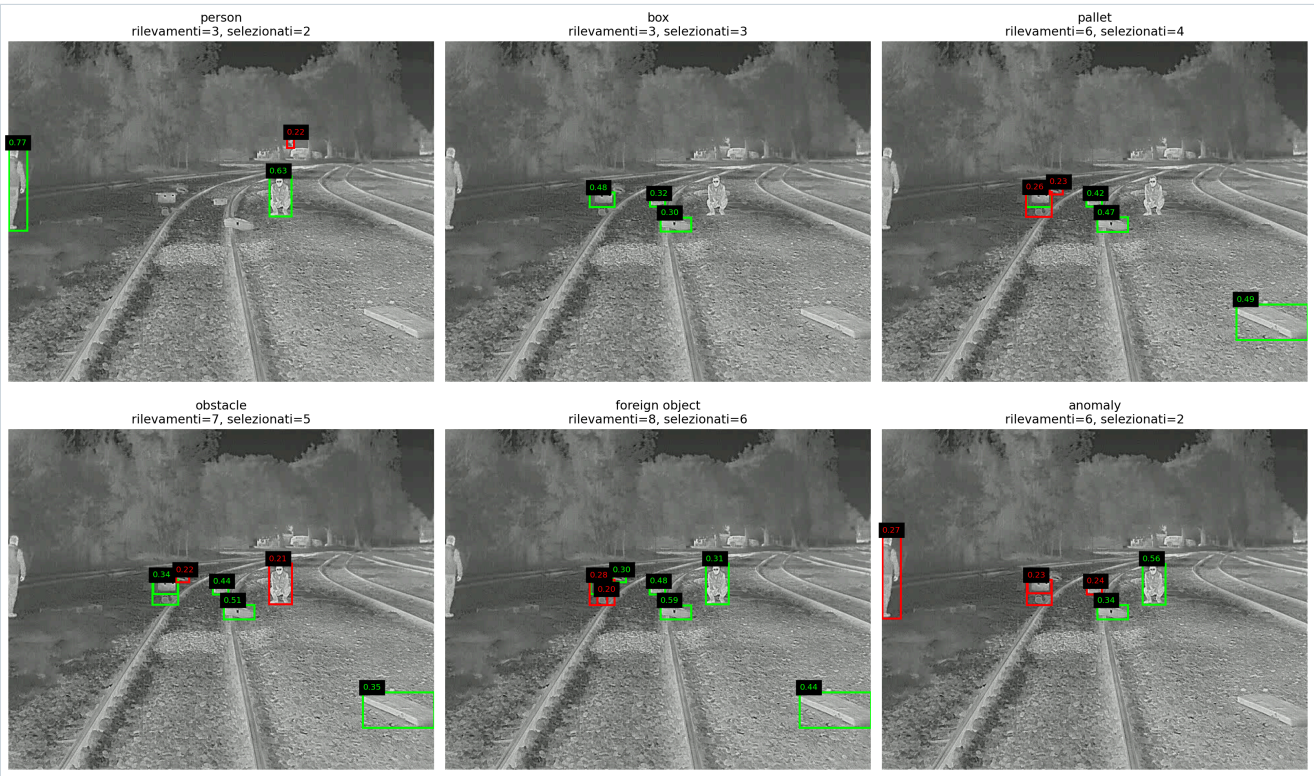


Figura 5 — Verde: box sopra la soglia 0,30. Rosso: rilevamenti deboli. I prompt generici producono regioni più eterogenee.

`person` conserva le due persone visibili. Il prompt `anomaly` seleziona la persona centrale e un oggetto, ma scarta la persona al bordo sinistro. `obstacle` e `foreign object` generano più candidati da verificare.

Il numero di box non è precision, recall o accuratezza: E03 usa un solo frame e non dispone di ground truth distinta per ciascuna classe. Inoltre la parola “anomaly” non trasforma un detector di oggetti in un metodo di anomaly detection.

8. Passaggio alla vera anomaly detection

Dopo E03 è stata esplicitata una correzione metodologica importante: l’obiettivo non è costruire una lista sempre più lunga di oggetti noti. Il sistema deve apprendere o rappresentare la normalità della scena e misurare lo scostamento di un nuovo

frame o di una sua regione.



Grounding DINO potrà rimanere come spiegazione semantica opzionale; SAM2 potrà rifinire o seguire una regione già considerata anomala. La decisione primaria, però, deve provenire dallo scostamento dalla normalità.

9. Preparazione E04 one-class

Preparato — non ancora eseguito

9.1 Costruzione dello split

È stato creato `build_anomaly_dataset.py`. Lo script campiona i due video annotati a 2 fps. Un frame è normale se non contiene box CVAT attive; è anomalo se contiene almeno una box attiva. Per evitare confini temporali incerti vengono esclusi i campioni la cui finestra di ± 2 secondi attraversa una transizione normale/anomalo.

Video	Train normale	Validation normale	Test anomalo	Esclusi
ir_2025-01-31_05-09-12.mp4	61	15	512	8
test_video_san_donato.mp4	108	27	383	8
Totale	169	42	895	16

Il dataset contiene 1.106 immagini JPEG e 3.473 box CVAT relative ai frame anomali di test. Le box non entrano nel training. Il pacchetto verificato per Colab è `anomaly_pilot_v3_colab.zip`, circa 100 MB, SHA-256:

```
9457b88634e4aad5735360c5ad68566c64f0ad23e0f6bb0c12d6aefb2988828f
```

9.2 Metodo previsto

E04 implementa una baseline one-class a feature patch ispirata al principio di PatchCore, ma non viene presentata come riproduzione integrale del metodo.

1. Le immagini vengono ridimensionate a 224×280 e normalizzate con statistiche ImageNet.
2. Una ResNet18 pre-addestrata e congelata estrae la feature map fino a `layer3`.
3. Ogni posizione della feature map diventa un vettore locale normalizzato.
4. I vettori dei 169 frame normali formano una memoria, ridotta al massimo a 30.000 patch con seme 42.
5. Per ogni patch di validation/test si calcola la distanza euclidea dalla patch normale più vicina.
6. Le distanze locali formano la heatmap; il 99° percentile produce lo score del frame.
7. La soglia normale/anomalo è il 95° percentile degli score dei soli 42 frame normali di validation.

$$a(p) = \min_{m \in M} \|f(p) - m\|_2$$

$$A(\text{frame}) = \text{percentile}_{99} \{ a(p) \}$$

Dove M è la memoria delle patch normali, $f(p)$ è la feature della patch p , $a(p)$ è lo score locale e $A(\text{frame})$ lo score globale.

9.3 Metriche previste

- AUROC e Average Precision a livello di frame;
- precision, recall e F1 alla soglia scelta sulla validation normale;
- matrice TN/FP/FN/TP;
- heatmap qualitative;
- *pointing game*: il massimo della heatmap deve cadere dentro almeno una box CVAT.

Alla data del documento E04 non è stato eseguito. Non esistono ancora AUROC, AP, F1 o heatmap E04 da riportare come risultati.

10. Spiegazione precisa del codice Python

Tutti gli script usano una struttura comune: funzioni riutilizzabili, parser CLI con `argparse`, gestione esplicita degli errori e funzione `main()` che restituisce 0 in caso di successo e 2 in caso di input non valido. Le directory di output vengono create solo quando necessario.

10.1 Script per video e clip

`src/video_inventory.py`

Funzione	Responsabilità
<code>decode_fourcc()</code>	Converte il codice numerico FourCC di OpenCV in una stringa leggibile, es. <code>h264</code> .
<code>inspect_video()</code>	Apre un video, legge FPS, frame count, larghezza, altezza e codec; calcola <code>durata = frame_count / fps</code> ; registra l'errore senza interrompere l'inventario.
<code>find_videos()</code>	Cerca ricorsivamente estensioni MP4, AVI, MOV e MKV.
<code>build_inventory()</code>	Ispeziona tutti i video e scrive il CSV con le colonne definite in <code>FIELDNAMES</code> .

```
python src/video_inventory.py \  
--input data/raw \  
--output results/inventory/inventario_video.csv
```

Il campo `ok=False` consente di conservare anche la riga di un video illeggibile, insieme al messaggio nel campo `error`.

`src/extract_frames.py`

`extraction_times()` genera timestamp nell'intervallo semiaperto `[start, end)`, separati da `1 / target_fps`. `extract_frames()` converte ogni timestamp nell'indice sorgente con `round(time × source_fps)`, evita duplicati, legge il frame e salva JPEG e indice CSV.

```
python src/extract_frames.py \  
--video data/raw/test_video_san_donato.mp4 \  
--out data/frames/san_donato \  
--fps 1 --start 10 --end 30
```

Lo script valida FPS, intervallo e presenza del file. Non consente una frequenza di estrazione superiore a quella del video. Il file `frames.csv` mantiene il collegamento tra immagine, frame originale e timestamp.

`src/make_clip.py`

`make_clip()` converte start/end in indici, posiziona `VideoCapture` al primo frame e scrive l'intervallo `[start, end)` con FPS e risoluzione originali. Usa `mp4v` per MP4 e `XVID` per AVI. OpenCV non conserva l'audio.

```
python src/make_clip.py \  
--video data/raw/test_video_san_donato.mp4 \  
--start 85 --end 105 \  
--out data/clips/san_donato_pilot_85_105.mp4
```

10.2 Script CVAT e visualizzazione

`src/inspect_cvat_export.py`

Usa `xml.etree.ElementTree`. `local_name()` rimuove eventuali namespace XML. `inspect_xml()` conta immagini, track, box, poligoni e maschere, distinguendo label configurate e label effettivamente annotate. `save_summary()` produce sia JSON sia CSV per rendere il controllo leggibile e riutilizzabile.

`src/cvat_to_boxes.py`

`convert_cvat_xml()` supporta due strutture CVAT: annotazioni video in `<track>` e annotazioni di immagini in `<image>`. Per ogni `<box>`, `box_row()` converte le coordinate in float e conserva `outside` e `occluded`. Le righe vengono ordinate per

sorgente, frame e track.

```
python src/cvat_to_boxes.py \
--xml data/exports_cvat/TASK/annotations.xml \
--out results/annotations/video_boxes.csv
```

`src/visualize_boxes.py`

Lo script carica soltanto box attive: `outside` $\notin$ `{1, true, yes}`. Normalizza frame come `1`, `1.0` o stringa, usa `frames.csv` quando disponibile e, in fallback, interpreta `frame_000001` come frame 0. Le coordinate vengono arrotondate e limitate ai bordi dell'immagine prima di disegnare rettangolo e label con OpenCV.

10.3 Metriche e valutazione

`src/metrics.py`

`box_area()` restituisce area zero per coordinate invertite. `iou()` calcola intersezione e unione:

$$\text{IoU} = \text{area}(B_{\text{pred}} \cap B_{\text{gt}}) / \text{area}(B_{\text{pred}} \cup B_{\text{gt}})$$

`match_boxes()` costruisce tutte le coppie predizione-ground truth, le ordina per IoU decrescente e applica matching greedy uno-a-uno sopra soglia. Ogni box può essere associata una sola volta.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{F1} = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$$

Le divisioni per zero restituiscono 0. `evaluate_frame()` combina matching e metriche per un singolo frame.

`src/evaluate_predictions.py`

`prepare_data()` valida le colonne, scarta box CVAT `outside`, converte score e coordinate, elimina predizioni sotto soglia e rifiuta box con `xbr ≤ xtl` o `ybr ≤ ytl`. La valutazione predefinita è class-agnostic, coerente con la label unica `anomaly`.

`evaluate_subset()` esegue il matching per frame e aggrega TP, FP e FN. `evaluation_rows()` crea righe globali e raggruppate per metodo, prompt e coppia metodo/prompt. Il CSV di predizione richiesto contiene:

```
frame,label,score,xtl,ytl,xbr,ybr,prompt,method
```

Le metriche box misurano la localizzazione rispetto alla ground truth; non bastano da sole a dimostrare anomaly detection se il metodo usa soltanto classi note di oggetti.

10.4 Configurazioni, prompt e stub SAM2

`configs/experiment_config.yaml`

Centralizza label principale, attributi, directory e soglie IoU 0,30 e 0,50. Separare configurazione e codice rende gli esperimenti documentabili senza modificare le funzioni.

`src/prompt_strategy.py` e `configs/prompts_zero_shot.yaml`

`load_prompts()` usa `yaml.safe_load`, richiede una mappa e valida che ciascuna famiglia sia una lista di stringhe non vuote. Le famiglie sono prompt diretti, prompt dello sfondo e prompt combinati. Dopo la correzione metodologica, queste liste sono considerate strumenti esplorativi o ausiliari, non la definizione dell'anomalia.

`src/sam2_notes_or_stub.py`

Lo stub usa `importlib.util.find_spec` per verificare la disponibilità di `torch` e `sam2` senza importarli. Se mancano, mostra istruzioni chiare e termina con successo. In questo modo la pipeline leggera funziona anche senza GPU e checkpoint.

10.5 Spiegazione dettagliata di `build_anomaly_dataset.py`

È lo script centrale per E04. Produce uno split one-class riproducibile e impedisce di usare accidentalmente annotazioni anomale nel training.

Funzione	Algoritmo e controlli
<code>VideoInfo</code>	Dataclass immutabile con nome, path, FPS e numero di frame.
<code>sample_frame_indices()</code>	Calcola la durata, genera indici a frequenza costante con arrotondamento, rimuove duplicati e controlla che $0 < \text{sample_fps} \leq \text{source_fps}$.
<code>classify_sampled_frames()</code>	Per ogni frame esamina una finestra temporale. È "normal" se nessun indice della finestra è anomalo; "anomaly" se tutti gli indici sono anomali; altrimenti è "ambiguous" ed escluso.
<code>split_normal_frames()</code>	Divide cronologicamente i frame normali 80/20; garantisce almeno un elemento in entrambi gli insiemi.
<code>read_inventory()</code>	Accetta soltanto righe con <code>ok=True</code> e costruisce la mappa nome → metadati.
<code>read_active_anomaly_frames()</code>	Legge tutti i CSV <code>*_boxes.csv</code> , considera attive le righe non <code>outside</code> , crea l'insieme dei frame anomali e conserva le box per la sola valutazione.
<code>save_selected_frames()</code>	Decodifica ogni video una sola volta in ordine sequenziale, salva solo gli indici richiesti con qualità JPEG configurabile e segnala eventuali frame mancanti.
<code>build_dataset()</code>	Combina tutte le fasi, crea cartelle train/validation/test, manifest, riepilogo, box e metadati. Rifiuta una cartella di output già esistente per evitare sovrascritture non tracciate.

```
python src/build_anomaly_dataset.py \
--inventory results/inventory/inventario_video.csv \
--annotations-dir results/annotations \
--out data/processed/anomaly_pilot_v3 \
--sample-fps 2 \
--margin-seconds 2 \
--train-fraction 0.8 \
--jpeg-quality 90
```

I file generati sono:

- `manifest.csv`: path, split, label, video, frame, timestamp e sorgente CVAT;
- `split_summary.csv`: conteggi per video e split;
- `ground_truth_boxes.csv`: 3.473 box attive dei frame di test;
- `metadata.json`: definizioni, parametri e conteggi;
- immagini sotto `train/normal`, `validation/normal` e `test/anomaly`.

11. Spiegazione dei notebook Google Colab

11.1 sam2_pilot_colab.ipynb — E01

1. Controllo GPU con `nvidia-smi`.
2. Installazione del repository SAM2 e caricamento del checkpoint Small.
3. Upload della clip, estrazione dei 500 frame in una directory temporanea.
4. Costruzione del video predictor e inizializzazione dello stato.
5. Inserimento della box manuale al frame 0 e selezione dell'oggetto.
6. Propagazione della maschera lungo il video.
7. Conversione maschera → box, salvataggio di CSV e NPZ.
8. Creazione del video overlay e di una tavola con sei frame.
9. Esportazione di metadati e ZIP.

11.2 grounded_sam2_pilot_colab.ipynb — E02

1. Caricamento della stessa clip e del primo frame.
2. Caricamento di `AutoProcessor` e `AutoModelForZeroShotObjectDetection`.
3. Inferenza con prompt `person` e soglie 0,20/0,15.
4. Filtro delle box sotto 0,30.
5. Passaggio delle due box mantenute a SAM2 come due identità.
6. Propagazione su 500 frame, overlay con colori distinti e salvataggio.

Nel runtime verificato erano presenti Torch 2.11.0+cu128, torchvision 0.26.0+cu128, transformers 4.57.0 e accelerate 1.14.0. Il warning di dipendenza Gradio/Pydantic non ha impedito gli import usati nell'esperimento.

11.3 Codice E03

La cella E03 esegue i sei prompt sullo stesso frame, registra tutte le box sopra 0,20, marca come selezionate quelle sopra 0,30, misura il tempo e genera figura, CSV dettagliato, riepilogo e metadati. Un reset del runtime aveva eliminato `gdino_processor`; il modello e il processor sono stati ricaricati, quindi la cella è stata rieseguita con successo.

11.4 e04_one_class_anomaly_colab.ipynb — E04

Cella	Funzione
1	Upload e decompressione di <code>anomaly_pilot_v3_colab.zip</code> .
2	Import, verifica manifest/box e controllo GPU.
3	ResNet18 congelata fino a layer3; preprocessing 224×280.
4	Estrazione delle patch normali e memoria massima di 30.000 vettori.
5	Distanza nearest-neighbor con <code>torch.cdist</code> ; heatmap e score al 99° percentile.
6	Soglia al 95° percentile normale; AUROC, AP, precision, recall, F1 e pointing game.
7	Tavola qualitativa con heatmap e box CVAT verdi.
8	Esportazione di score, mappe, memoria, runtime, figura e ZIP.

La memoria salvata in FP16 consente di ricostruire il riferimento normale. `frame_scores.csv` conserva uno score per ogni frame, `anomaly_maps.npz` le mappe locali e `summary.json` le metriche. Il codice non usa Grounding DINO per la decisione di anomalia.

12. Test, controlli e artefatti salvati

12.1 Test del codice

- `test_metrics.py`: IoU, matching e precision/recall/F1;
- `test_cvat.py`: parsing e conversione di export CVAT;
- `test_anomaly_dataset.py`: campionamento, margini e split cronologico.

Il nuovo script E04 è stato compilato con `py_compile` e le asserzioni essenziali sono state eseguite con successo. Nell'ambiente locale corrente `pytest` non è installato, quindi l'intera suite dovrà essere rilanciata dopo aver aggiunto la dipendenza di sviluppo.

12.2 Controlli sui dati E04

- 1.106 righe del manifest e 1.106 immagini presenti;
- 169 train normal, 42 validation normal, 895 test anomaly;
- 3.473 box CVAT di test;
- metadati JSON coerenti con i file;
- ZIP verificato senza errori con `unzip -t`.

12.3 Artefatti principali

Area	File/cartella	Contenuto
Inventario	results/inventory/	Metadati tecnici dei tre video
Ground truth	results/annotations/	Due CSV CVAT convertiti
E01	results/qualitative/sam2_pilot_01/	Video, maschere, box, figure, JSON e ZIP
E01 metriche	results/metrics/sam2_pilot_01/	IoU per frame e riepilogo
E02	results/qualitative/grounded_sam2_pilot_02/	Detection, tracking, video, maschere e metadati
E02 metriche	results/metrics/grounded_sam2_pilot_02/	Matching candidati-track CVAT
E03	results/qualitative/grounding_dino_prompt_sensitivity_e03/	Figura, detection, summary, metadati e analisi
E04 dati	data/processed/anomaly_pilot_v3/	Split one-class e box di valutazione
E04 Colab	notebooks/e04_one_class_anomaly_colab.ipynb	Notebook completo non ancora eseguito

13. Limiti attuali e rischi metodologici

1. **Ground truth da revisionare.** Track interpolate male alterano IoU, F1 e qualsiasi valutazione di localizzazione.
2. **Terzo video non annotato.** Non è ancora disponibile un test aggiuntivo indipendente.
3. **Definizione automatica di normalità.** E04 interpreta “nessuna box CVAT attiva” come normale; questi frame vanno controllati visivamente per escludere omissioni.
4. **Training e test nelle stesse scene.** Il pilot misura anomalie contestuali rispetto alle telecamere note; non prova generalizzazione a una nuova stazione.
5. **Feature RGB su termico.** ResNet18 è pre-addestrata su immagini RGB e può subire domain shift sulle immagini termiche.
6. **Tempo.** E03 misura il primo prompt con warm-up; i tempi non costituiscono ancora un benchmark prestazionale.
7. **E01-E03 non sono anomaly detection completa.** Devono essere presentati come studi preliminari dei moduli di localizzazione.

14. Prossimi passi ordinati

1. **Eseguire E04** in Colab e scaricare lo ZIP completo dei risultati.

- 2. **Controllare le heatmap**: verificare se evidenziano persone/oggetti o cambiamenti irrilevanti dello sfondo.
- 3. **Archiviare E04** nel registro con AUROC, AP, F1 e pointing game reali.
- 4. **Correggere CVAT** nell'intervallo 98-101 s e negli altri errori visibili.
- 5. **Ricalcolare E01/E02** con ground truth corretta.
- 6. **Annotare il terzo video** e usarlo come test aggiuntivo o hold-out.
- 7. **Confrontare una seconda baseline** solo se il tempo lo permette, mantenendo identico lo split.
- 8. **Integrare SAM2 dopo la heatmap**, se utile, per rifinire e propagare la regione anomala.
- 9. **Scrivere i capitoli** usando registro, figure, tabelle e limiti già salvati.

Il prossimo passo operativo immediato è aprire `e04_one_class_anomaly_colab.ipynb`, attivare la GPU e caricare `anomaly_pilot_v3_colab.zip`.

15. Materiale utilizzabile nella tesi e nelle slide

Messaggio	Materiale consigliato
Problema e dominio termico	Un frame rappresentativo autorizzato e la definizione di anomalia contestuale
Ground truth	Protocollo CVAT, box e spiegazione di keyframe/interpolazione
Ruolo di SAM2	Figura 2 e distinzione prompt → segmentazione/tracking
Pipeline automatica ausiliaria	Figure 3 e 4: testo → box → SAM2
Sensibilità ai prompt	Figura 5, dichiarando che non è anomaly detection finale
Importanza della ground truth	Figura 1 e metriche provvisorie
Vera anomaly detection	Schema E04: normalità → distanza → score/heatmap
Risultati finali E04	Da aggiungere dopo l'esecuzione, senza anticipare numeri

Il materiale è già organizzato in `docs/scaletta_presentazione.md` per una presentazione di circa 8-10 minuti. Le figure E01-E03 possono essere usate nelle slide purché l'autorizzazione sui dati lo consenta.

16. Riferimenti essenziali

- 1. K. Roth et al., “Towards Total Recall in Industrial Anomaly Detection”, CVPR 2022. Metodo PatchCore, riferimento concettuale per la memoria di feature locali normali.
https://openaccess.thecvf.com/content/CVPR2022/html/Roth_Towards_Total_Recall_in_Industrial_Anomaly_Detection_CVPR_2022_paper.html
- 2. N. Ravi et al., “SAM 2: Segment Anything in Images and Videos”, 2024. Repository ufficiale Meta.
<https://github.com/facebookresearch/sam2>
- 3. S. Liu et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection”, 2023.
<https://github.com/IDEA-Research/GroundingDINO>
- 4. C. Li et al., “Segmenting Objects in Day and Night: Edge-Conditioned CNN for Thermal Image Semantic Segmentation”, IEEE TNNLS, DOI 10.1109/TNNLS.2020.3009373.
- 5. Thermal Anomaly Detection Dataset, Kaggle. Benchmark esterno termico da trattare dichiarando il domain gap rispetto al contesto ferroviario.
<https://www.kaggle.com/datasets/neelu1/thermal-anomaly-detection-dataset>